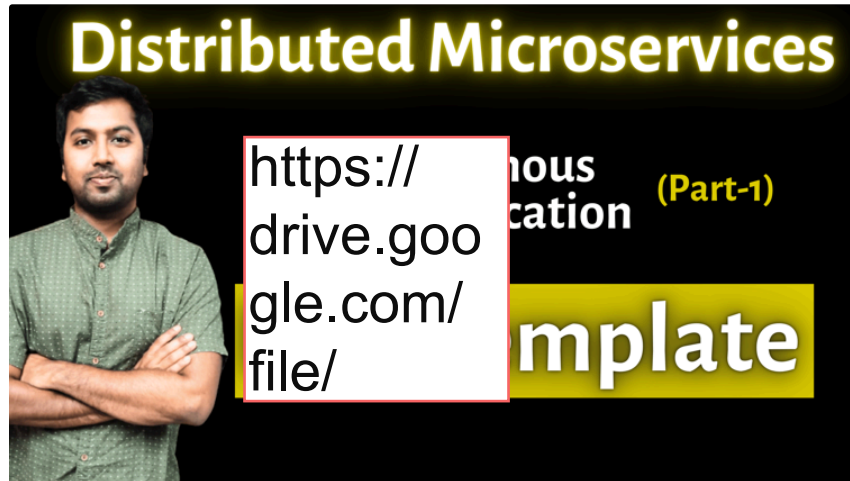


RestClient - Synchronous Communication

Prerequisite to understand RestClient is, to first understand its predecessor RestTemplate



Kindly check the Part-1 (RestTemplate), if there is any doubt with that topic.

In previous video, we saw the Limitation of RestTemplate:

- In RestTemplate, there are already so many overloaded methods, so its hard to remember and maintain.
- RestTemplate was build before concepts like Retry, circuit breaker etc.. So adding support means more overloaded methods and not user friendly.
- RestTemplate is in Maintenance mode - means no new feature, only bug fixes.

Alternative of RestTemplate: **RestClient**

- Introduced in Spring Framework 6.0+ and SpringBoot 3.0+
- Synchronous/blocking in nature, means client wait for the response from the Server before continuing.
- Modern, fluent based API that is more readable and easy to maintain.

OrderService needs to invoke ProductService

OrderService (port 8081) -----> **ProductService** (port 8082)

```
1 @Configuration
2 public class AppConfig {
3
4     @Bean
5     public RestClient restClientInstance() {
6         return RestClient.create();
7     }
8 }
9
```

```

1 @RestController
2 @RequestMapping("/products")
3 public class ProductController {
4
5     @GetMapping("/{id}")
6     public String getProduct(@PathVariable String id) {
7         return "Product fetched with id: " + id;
8     }
9 }
10

```

```

1 @RestController
2 @RequestMapping("/orders")
3 public class OrderController {
4
5     @Autowired
6     RestClient restClient;
7
8     @GetMapping("/{id}")
9     public ResponseEntity<String> getOrder(@PathVariable String id) {
10
11         String response = restClient
12             .get()
13             .uri("http://localhost:8082/products/" + id)
14             .retrieve()
15             .body(String.class);
16
17         System.out.println(
18             "Response from Product API called from order service:
19 " + response
20         );
21         return ResponseEntity.ok("order call successful");
22     }
23 }
24

```

Concept & Coding

GET
localhost:8081/orders/1

Params
Authorization
Headers (6)
Body
Scripts
Settings

Query Params

	Key	Value
	Key	Value

Body
Cookies
Headers (5)
Test Results

Raw
Preview
Visualize

1 order call successful

```

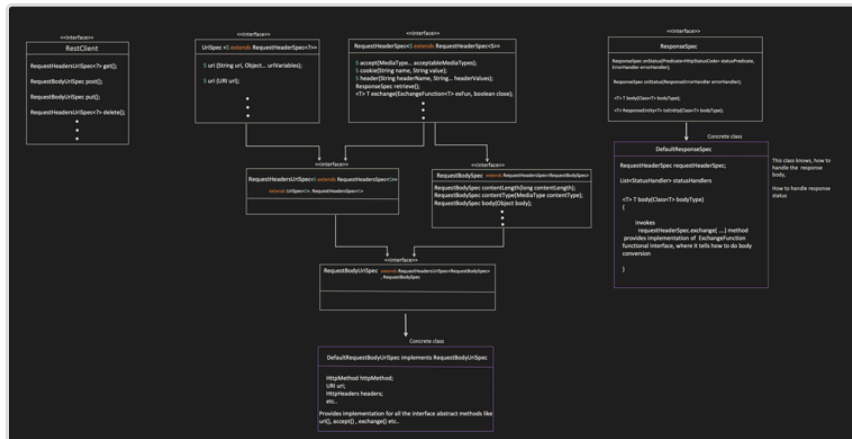
2025-05-27T15:14:57.352+05:30 INFO 66070 --- [main] c.c.o.OrderserviceApplication : Starting OrderserviceApplication using J
2025-05-27T15:14:57.353+05:30 INFO 66070 --- [main] c.c.o.OrderserviceApplication : No active profile set, falling back to 1
2025-05-27T15:14:57.700+05:30 INFO 66070 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8081 (http)
2025-05-27T15:14:57.705+05:30 INFO 66070 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2025-05-27T15:14:57.705+05:30 INFO 66070 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/
2025-05-27T15:14:57.722+05:30 INFO 66070 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicat
2025-05-27T15:14:57.722+05:30 INFO 66070 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initializati
2025-05-27T15:14:57.986+05:30 INFO 66070 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8081 (http) with
2025-05-27T15:14:57.910+05:30 INFO 66070 --- [main] c.c.o.OrderserviceApplication : Started OrderserviceApplication in 0.713
2025-05-27T15:15:04.840+05:30 INFO 66070 --- [nio-8081-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'd
2025-05-27T15:15:04.840+05:30 INFO 66070 --- [nio-8081-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2025-05-27T15:15:04.841+05:30 INFO 66070 --- [nio-8081-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 0 ms
Response from Product API called from order service: Product fetched with id: 1

```

So, first important thing to understand is, how this Fluent API works?

- Fluent API means chaining of method calls.

- Each method call returns an object (of next step) that exposes next set of operations (methods).



I am assuming that, we all know about Generics Class and Wildcards in Java



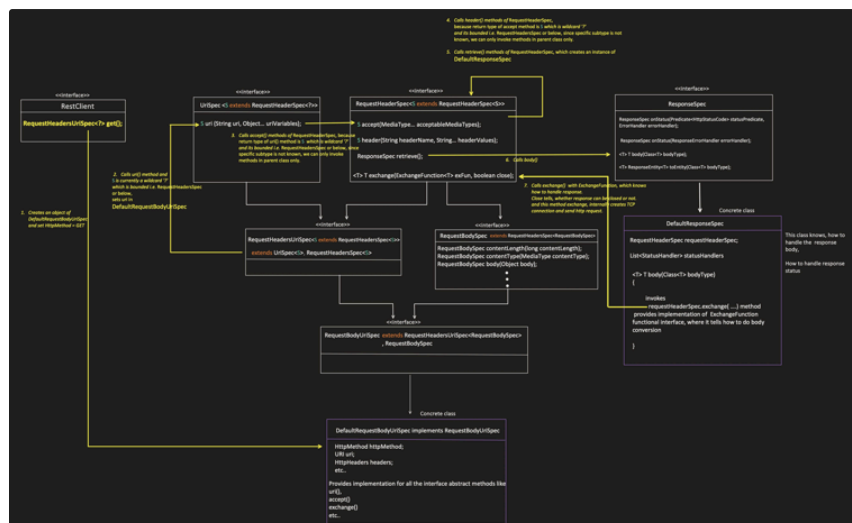
Now, lets see how Fluent API works for GET, POST, DELETE operation.

Concept & Coding

GET:

```

1 RestClient restClient = RestClient.create();
2
3 String response = restClient
4     .get()
5     .uri("http://localhost:8082/products/" + id)
6     .accept(MediaType.APPLICATION_JSON)
7     .header("X-Custom-Header", "xyz")
8     .retrieve()
9     .body(String.class);
10
11
  
```



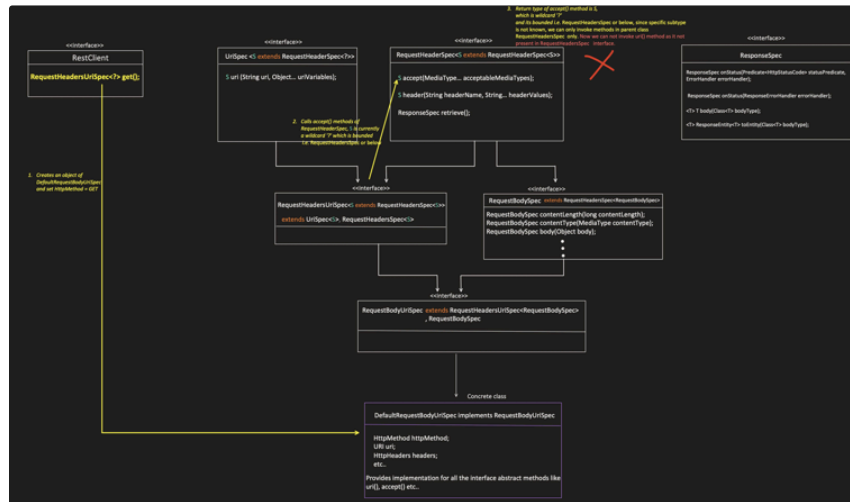
Let see, how flow can be broken

GET

```
RestClient restClient = RestClient.create();
```

```
String response = restClient  
    .get()  
    .accept(MediaType.APPLICATION_JSON)  
    .uri_
```

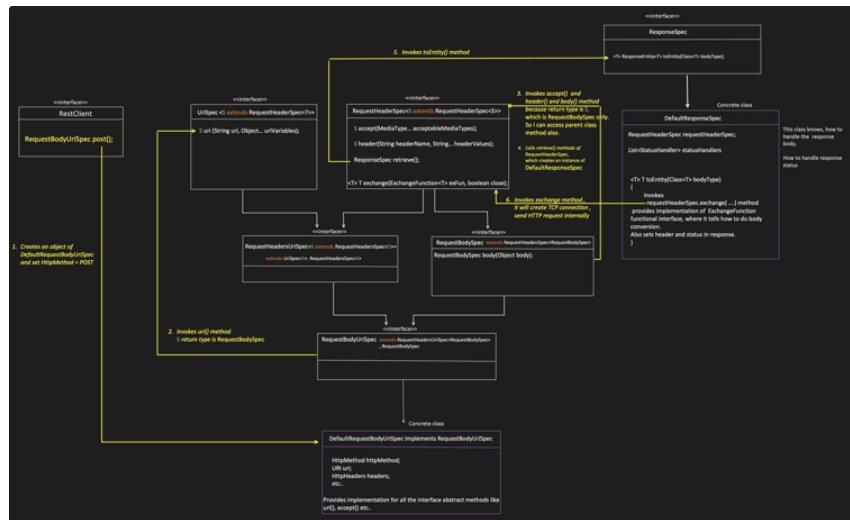
Instead of uri method first, I invoked the accept method first and now I can not set the URI itself. Why, does sequence of call matters?



Concept & Coding

POST:

```
1 RestClient restClient = RestClient.create();  
2  
3 ResponseEntity<ProductEntity> response = restClient  
4     .post()  
5     .uri("http://localhost:8082/products/create")  
6     .accept(MediaType.APPLICATION_JSON)  
7     .header("Content-Type", "application/json")  
8     .body(new ProductEntity("Ice-cream")) // some new object which  
9     need to be created  
10    .retrieve()  
11    .toEntity(ProductEntity.class);  
12 ProductEntity responseBody = response.getBody();  
13
```

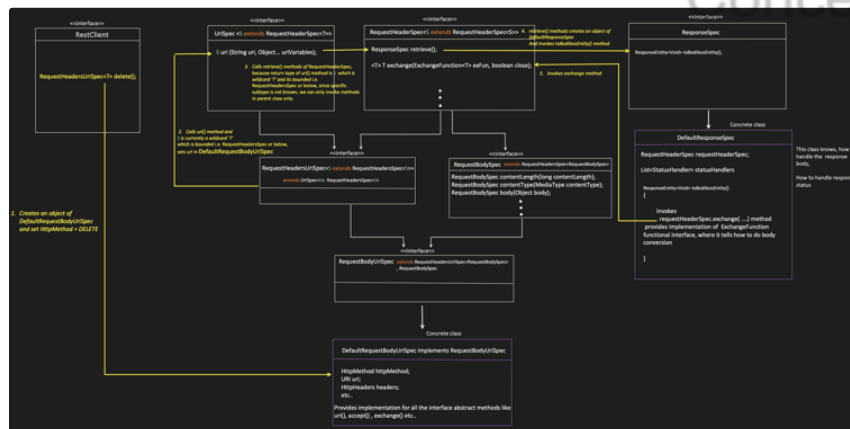


DELETE:

```

1 RestClient restClient = RestClient.create();
2
3 ResponseEntity<Void> response = restClient
4   .delete()
5   .uri("http://localhost:8082/products/" + id)
6   .retrieve()
7   .toBodilessEntity();
8
9 HttpStatus deletionStatus = response.getStatusCode();
10

```



Exception Handling:

```

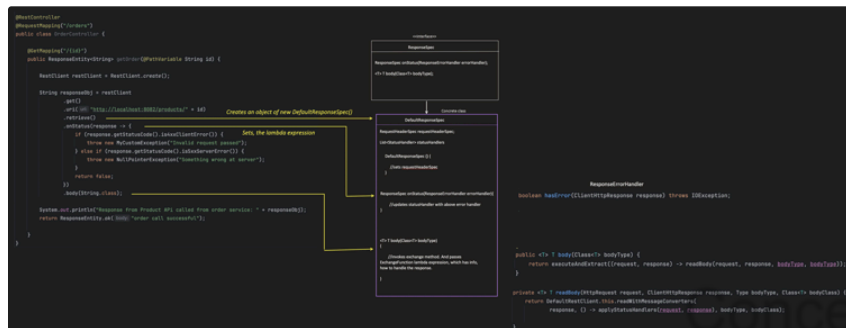
1 @RestController
2 @RequestMapping("/orders")
3 public class OrderController {
4
5   @GetMapping("/{id}")
6   public ResponseEntity<String> getOrder(@PathVariable String id) {
7
8     RestClient restClient = RestClient.create();
9
10    String responseObj = restClient
11      .get()
12      .uri("http://localhost:8082/products/" + id)
13      .retrieve()
14      .onStatus(response -> {

```

```

15
16         if (response.getStatusCode().is4xxClientError()) {
17             throw new MyCustomException("Invalid request
passed");
18         } else if
(response.getStatusCode().is5xxServerError()) {
19             throw new NullPointerException("Something
wrong at server");
20         }
21
22         return false;
23     })
24     .body(String.class);
25
26     System.out.println(
27         "Response from Product API called from order service:
" + responseObj
28     );
29
30     return ResponseEntity.ok("order call successful");
31 }
32 }
33

```



If we want full control over Response building and Exception handling

Then we can directly call exchange method, instead of relying of ResponseSpec to do that.

```

1  @RestController
2  @RequestMapping("/orders")
3  public class OrderController {
4
5      @GetMapping("/{id}")
6      public ResponseEntity<String> getOrder(@PathVariable String id) {
7
8          RestClient restClient = RestClient.create();
9
10         String responseObj = restClient
11             .get()
12             .uri("http://localhost:8082/products/" + id)
13             .exchange((request, response) -> {
14
15                 if (response.getStatusCode().is4xxClientError()) {
16                     throw new MyCustomException("Invalid request
passed");
17                 } else if
(response.getStatusCode().is5xxServerError()) {
18                     throw new InternalError("Something wrong at
server");
19                 } else {
20                     // response mapping logic if no error
21                     return StreamUtils.copyToString(
22                         response.getBody(),
23                         StandardCharsets.UTF_8
24                     );
25                 }
26             });
27     }
28 }
29

```

```

25         }
26     });
27
28     System.out.println(
29         "Response from Product API called from order service:
30     " + responseObj
31     );
32     return ResponseEntity.ok("order call successful");
33 }
34 }
35

```

exchange() method:

exchange() method:

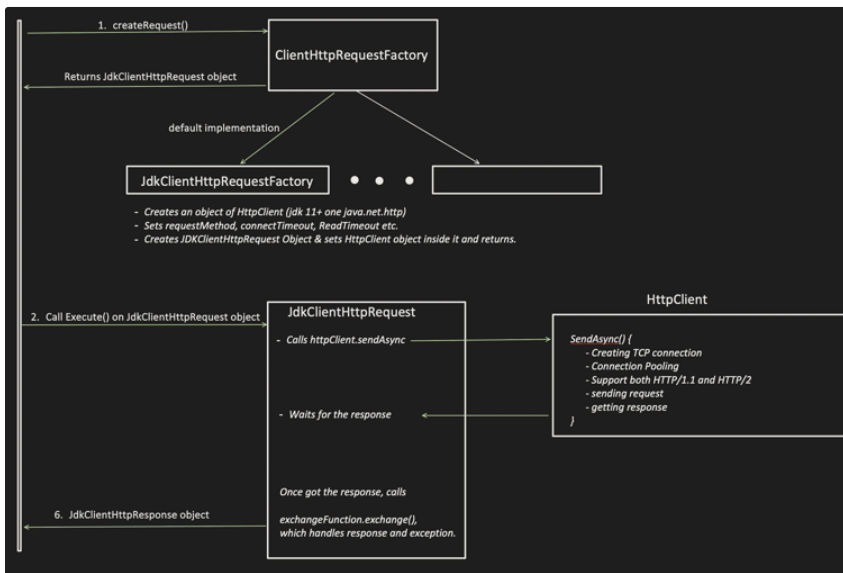
```

private <T> T exchangeInternal(ExchangeFunction<T> exchangeFunction, boolean close) {
    Assert.notNull(exchangeFunction, "message: ExchangeFunction must not be null");

    ClientHttpResponse clientResponse = null;
    Observation observation = null;
    Observation.Scope observationScope = null;
    URI uri = null;
    try {
        uri = initUri();
        String serializedCookies = serializeCookies();
        if (serializedCookies != null) {
            getHeaders().set(HttpHeaders.COOKIE, serializedCookies);
        }
        HttpHeaders headers = initHeaders();

        ClientHttpRequest clientRequest = createRequest(uri); — ①
        if (headers != null) {
            clientRequest.getHeaders().addAll(headers);
        }
        Map<String, Object> attributes = getAttributes();
        clientRequest.getAttributes().putAll(attributes);
        ClientRequestObservationContext observationContext = new ClientRequestObservationContext(clientRequest);
        observationContext.setUriTemplate((String) attributes.get(URI_TEMPLATE_ATTRIBUTE));
        observation = ClientHttpObservationDocumentation.HTTP_CLIENT_EXCHANGES.observation(observationConvention,
            DEFAULT_OBSERVATION_CONVENTION, () -> observationContext, observationRegistry).start();
        observationScope = observation.openScope();
        if (this.body != null) {
            this.body.writeTo(clientRequest);
        }
        if (this.httpRequestConsumer != null) {
            this.httpRequestConsumer.accept(clientRequest);
        }
        clientResponse = clientRequest.execute(); — ②
        observationContext.setResponse(clientResponse);
        ConvertibleClientHttpResponse convertibleWrapper = new DefaultConvertibleClientHttpResponse(clientResponse);
        return exchangeFunction.exchange(clientRequest, convertibleWrapper); — ③
    }
}

```



Adding Interceptors

OrderService:

```
1 public class MyCustomRequestInterceptor implements
  ClientHttpRequestInterceptor {
2
3     @Override
4     public ClientHttpResponse intercept(
5         HttpRequest request,
6         byte[] body,
7         ClientHttpRequestExecution execution
8     ) throws IOException {
9
10        request.getHeaders().add("X-Custom-Header", "myvalue");
11        return execution.execute(request, body);
12    }
13 }
14
15
```

```
1 @Configuration
2 public class AppConfig {
3
4     @Bean
5     public RestClient restClientInstance(
6         ClientHttpRequestInterceptor myCustomInterceptor
7     ) {
8         return RestClient.builder()
9             .requestInterceptor(myCustomInterceptor)
10            .build();
11    }
12
13     @Bean
14     public ClientHttpRequestInterceptor customRequestInterceptor() {
15         return new MyCustomRequestInterceptor();
16    }
17 }
18
```

```
1 @RestController
2 @RequestMapping("/orders")
3 public class OrderController {
4
5     @Autowired
6     RestClient restClient;
7
8     @GetMapping("/{id}")
9     public ResponseEntity<String> getOrder(@PathVariable String id) {
10
11         ResponseEntity<String> responseEntityObj = restClient
12             .get()
13             .uri("http://localhost:8082/products/" + id)
14             .retrieve()
15             .toEntity(String.class);
16
17         System.out.println(
18             "Response from Product API called from order service:
19
20             + responseEntityObj.getBody()
21         );
22     }
23 }
```



```

22         return ResponseEntity.ok("order call successful");
23     }
24 }
25

```

ProductService

```

1  @RestController
2  @RequestMapping("/products")
3  public class ProductController {
4
5      @GetMapping("/{id}")
6      public String getProduct(@PathVariable String id) {
7          return "Product fetched with id: " + id;
8      }
9  }
10

```

When product service got invoked, able to see the header which we added using interceptor

```

> this = {ProductController@5847}
> id = "1"
> headers = {LinkedHashMap@5845} size = 7
  > "connection" -> "Upgrade, HTTP2-Settings"
  > "content-length" -> "0"
  > "host" -> "localhost:8082"
  > "http2-settings" -> "AAEAAEAAAAIAAABAAMAAABKAAQBAAAAAAUAAEAA"
  > "upgrade" -> "h2c"
  > "user-agent" -> "Java-http-client/17.0.12"
  > "x-custom-header" -> "myvalue"

```

Interceptor invokes at step2

```

private <T> T exchangeInternal(ExchangeFunction<T> exchangeFunction, boolean close) {
    Assert.notNull(exchangeFunction, "message: 'ExchangeFunction must not be null'");

    ClientHttpResponse clientResponse = null;
    Observation observation = null;
    Observation.Scope observationScope = null;
    URI uri = null;
    try {
        uri = initUri();
        String serializedCookies = serializeCookies();
        if (serializedCookies != null) {
            getHeaders().set(HttpHeaders.COOKIE, serializedCookies);
        }
        HttpHeaders headers = initHeaders();

        ClientHttpRequest clientRequest = createRequest(uri); ——— ①
        if (headers != null) {
            clientRequest.getHeaders().addAll(headers);
        }
        Map<String, Object> attributes = getAttributes();
        clientRequest.getAttributes().putAll(attributes);
        ClientRequestObservationContext observationContext = new ClientRequestObservationContext(clientRequest);
        observationContext.setUriTemplate((String) attributes.get(URI_TEMPLATE_ATTRIBUTE));
        observation = ClientHttpObservationDocumentation.HTTP_CLIENT_EXCHANGES.observation(observationConvention,
            DEFAULT_OBSERVATION_CONVENTION, () -> observationContext, observationRegistry).start();
        observationScope = observation.openScope();
        if (this.body != null) {
            this.body.writeTo(clientRequest);
        }
        if (this.httpRequestConsumer != null) {
            this.httpRequestConsumer.accept(clientRequest);
        }
        clientResponse = clientRequest.execute(); ——— ②
        observationContext.setResponse(clientResponse);
        ConvertibleClientHttpResponse convertibleWrapper = new DefaultConvertibleClientHttpResponse(clientResponse);
        return exchangeFunction.exchange(clientRequest, convertibleWrapper); ——— ③
    }
}

```

Interceptor, will get invoked when this method is trying to execute, our execute call is wrapped in InterceptingClientHttpRequest

Concept && Coding